Stockora AI

Intelligent POS & Inventory Management Platform

Developer Guide

Complete Technical Reference for Engineers & Developers

Version 2.0.0 | PHP 8.x · MySQL 8.x · PDO | July 2025

Table of Contents

CHAPTER 1

Architecture Overview

Technology stack, project structure, request lifecycle, multi-tenant model

1.1 Technology Stack

Layer	Technology	Version	Role
Backend Language	PHP	8.0+	All server-side logic, routing, DB queries
Database	MySQL	8.0+	Relational storage with InnoDB engine
DB Abstraction	PDO	Built-in	Prepared statements, prevents SQL injection
Web Server	Apache / PHP-S	—	Serves PHP files; dev uses built-in PHP server
Frontend CSS	Bootstrap 5.3.2	5.3.2	Admin panel only (shop panel is custom CSS)
Frontend Icons	Bootstrap Icons	1.11+	Sidebar icons, action buttons
Charts	Chart.js	3.x / 4.x	Dashboard charts, analytics graphs
Drag & Drop	SortableJS	1.15+	Theme marketplace ordering
Process Manager	PM2	Latest	Keeps PHP dev server alive in background
Storefront CSS	100% Custom	—	CSS custom properties / design tokens only

1.2 Project Directory Structure

```
/home/user/stockora/ ← Root ├── admin/ ← Platform admin panel (superadmin only) │ ├── index.php Admin dashboard │ ├── shops.php Manage all shops │ ├── subscriptions.php Subscription CRUD │ ├── revenue.php Platform revenue reports │ └── settings.php Global platform settings ├── shop/ ← Shop owner panel (per-tenant) │ ├── index.php Shop dashboard with AI autopilot │ ├── pos.php POS billing screen │ ├── products.php Product CRUD │ ├── sales.php Sales history │ ├── store.php Public storefront │ ├── theme_marketplace.php Enterprise theme engine │ ├── store_customize.php 11-tab store config │ ├── ai_lab.php AI Product Engine & Brand Score │ └── analytics.php Analytics & reports ├── includes/ ← Shared core files │ ├── config.php DB credentials, constants, getDB() │ ├── functions.php All helper
```

```
functions | ├── shop_layout.php shopHeader() / shopFooter() | └── admin_layout.php adminHeader() / adminFooter() ├── assets/ ← Static assets | ├── css/ Custom CSS files | ├── js/ Custom JS modules | ├── images/ Static images | └── uploads/ User-uploaded files (logos, etc.) ├── api/ ← Dedicated API endpoints | ├── sales.php Sales data API | └── shop_report.php Shop report export API ├── docs/ ← Documentation files ├── store.php ← Public storefront entry point ├── index.php ← Root redirect (shop vs admin) ├── login.php ← Unified login page ├── logout.php ← Session destroy + redirect ├── database.sql ← Full schema + seed data └── ecosystem.config.cjs ← PM2 config (dev server)
```

1.3 Request Lifecycle

Every PHP page follows this exact boot sequence:

```php
// 1. Load core (ALWAYS first line) require_once '../includes/functions.php'; // 2. Enforce authentication guard requireShop(); // OR requireAdmin() for admin pages // 3. Get shop context (never from POST/GET) $shopId = (int)$_SESSION['shop_id']; // 4. Handle AJAX POST actions (early return with JSON) if ($_SERVER['REQUEST_METHOD'] === 'POST') { ob_clean(); // Clear any accidental output jsonResponse(['success' => true, 'data' => ...]); } // 5. Fetch page data (PDO prepared statements only) $db = getDB(); $stmt = $db->prepare("SELECT * FROM products WHERE shop_id=?"); $stmt->execute([$shopId]); // 6. Load layout header (outputs HTML head + nav) require_once '../includes/shop_layout.php'; shopHeader('Page Title', 'nav_key'); // 7. Output page HTML // ... your page content ... // 8. Load layout footer shopFooter();
```

1.4 Multi-Tenant Isolation Model

Stockora is a **shared-database multi-tenant** system. All shops share the same MySQL database, but every sensitive table has a `shop_id` column that acts as a tenant discriminator.

> **Critical Security Rule — NEVER Trust Input for shop_id**
>
> `$shopId` is ALWAYS read from `$_SESSION['shop_id']` — never from `$_POST`, `$_GET`, or URL parameters. Violating this rule allows horizontal privilege escalation (one shop accessing another shop's data).

Table	Tenant Column	Notes
products	shop_id	Always filter: WHERE shop_id = ?
categories	shop_id	Always filter: WHERE shop_id = ?
sales	shop_id	Always filter: WHERE shop_id = ?
customers	shop_id	Always filter: WHERE shop_id = ?

Table	Tenant Column	Notes
settings	shop_id	All shop config stored here as key-value pairs
users	shop_id	Staff members linked to one shop
online_orders	shop_id	Always filter: WHERE shop_id = ?

CHAPTER 2

Database Design

Setup, ER overview, core schemas, and settings key conventions

2.1 Setup & Connection

1. **Create Database**

   `mysql -u root -p -e "CREATE DATABASE stockora CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;"`

2. **Import Schema**

   `mysql -u root -p stockora < database.sql` — creates all tables and inserts default admin user.

3. **Update Credentials**

   Edit `includes/config.php` — set DB_HOST, DB_USER, DB_PASS, DB_NAME to match your environment.

2.2 Entity Relationship Overview

Parent Table	Child Tables	Relationship
shops	users, settings, products, categories, sales, customers, subscriptions, online_orders, purchases, expenses	One-to-Many (shop_id FK)
products	sale_items, purchase_items, online_order_items	One-to-Many
categories	products	One-to-Many
sales	sale_items	One-to-Many
customers	sales, credit_transactions	One-to-Many
online_orders	online_order_items	One-to-Many

2.3 Core Table Schemas

shops — Master tenant table

```
CREATE TABLE `shops` ( `id` INT(11) NOT NULL AUTO_INCREMENT, `name` VARCHAR(255) NOT NULL, `owner_name` VARCHAR(255) NOT NULL, `email` VARCHAR(255) NOT NULL, `phone` VARCHAR(50) DEFAULT NULL, `address` TEXT DEFAULT NULL, `logo` VARCHAR(500) DEFAULT NULL, `city` VARCHAR(100) DEFAULT NULL, `status` ENUM('active','inactive','suspended') NOT NULL DEFAULT 'active', `notes` TEXT DEFAULT NULL, `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP, `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP, PRIMARY KEY (`id`), UNIQUE KEY `uq_shops_email` (`email`) ) ENGINE=InnoDB;
```

settings — All per-shop configuration as key-value pairs

```
CREATE TABLE `settings` ( `id` INT(11) NOT NULL AUTO_INCREMENT, `shop_id` INT(11) NOT NULL, `setting_key` VARCHAR(100) NOT NULL, `setting_value` LONGTEXT DEFAULT NULL, `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP, PRIMARY KEY (`id`), UNIQUE KEY `uq_settings` (`shop_id`, `setting_key`), KEY `idx_settings_shop` (`shop_id`), CONSTRAINT `fk_settings_shop` FOREIGN KEY (`shop_id`) REFERENCES `shops` (`id`) ON DELETE CASCADE ) ENGINE=InnoDB;
```

products — Product catalog

```
CREATE TABLE `products` ( `id` INT(11) NOT NULL AUTO_INCREMENT, `shop_id` INT(11) NOT NULL, `category_id` INT(11) DEFAULT NULL, `name` VARCHAR(255) NOT NULL, `sku` VARCHAR(100) DEFAULT NULL, `barcode` VARCHAR(100) DEFAULT NULL, `description` TEXT DEFAULT NULL, `sale_price` DECIMAL(10,2) NOT NULL DEFAULT 0.00, `cost_price` DECIMAL(10,2) NOT NULL DEFAULT 0.00, `stock_qty` INT(11) NOT NULL DEFAULT 0, `reorder_level` INT(11) NOT NULL DEFAULT 5, `unit` VARCHAR(50) DEFAULT 'pcs', `expiry_date` DATE DEFAULT NULL, `image` VARCHAR(500) DEFAULT NULL, `status` ENUM('active','inactive') NOT NULL DEFAULT 'active', `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP, `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP, PRIMARY KEY (`id`), KEY `idx_products_shop` (`shop_id`), KEY `idx_products_category` (`category_id`), KEY `idx_products_sku` (`sku`), CONSTRAINT `fk_products_shop` FOREIGN KEY (`shop_id`) REFERENCES `shops` (`id`) ON DELETE CASCADE ) ENGINE=InnoDB;
```

online_orders — Commerce Cloud orders

```
CREATE TABLE `online_orders` ( `id` INT(11) NOT NULL AUTO_INCREMENT, `shop_id` INT(11) NOT NULL, `order_no` VARCHAR(50) NOT NULL, `customer_name` VARCHAR(255) NOT NULL, `customer_phone` VARCHAR(50) NOT NULL, `customer_email` VARCHAR(255) DEFAULT NULL, `delivery_address` TEXT NOT NULL, `items_json` LONGTEXT NOT NULL, -- JSON array of ordered items `subtotal` DECIMAL(10,2) NOT NULL DEFAULT 0.00, `discount` DECIMAL(10,2) NOT NULL DEFAULT 0.00, `delivery_fee` DECIMAL(10,2) NOT NULL DEFAULT 0.00, `total` DECIMAL(10,2) NOT NULL DEFAULT 0.00, `payment_method` VARCHAR(50) DEFAULT 'cod', `status` ENUM('pending','confirmed','shipped','delivered','cancelled') NOT NULL DEFAULT 'pending', `notes` TEXT DEFAULT NULL, `created_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP, `updated_at` DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP, PRIMARY KEY (`id`), KEY `idx_orders_shop` (`shop_id`), KEY `idx_orders_status` (`status`), CONSTRAINT `fk_orders_shop` FOREIGN KEY (`shop_id`) REFERENCES `shops` (`id`) ON DELETE CASCADE ) ENGINE=InnoDB;
```

2.4 Settings Key Naming Convention

All shop configuration lives in the `settings` table. Keys follow a prefix-based naming convention:

Prefix	Example Keys	Value Type
`store_`	store_theme, store_slug, store_status, store_name	String / JSON
`theme_`	theme_favourites, theme_backups, theme_version_history	JSON array
`shop_`	shop_logo, shop_phone, shop_address	String
`seo_`	seo_title, seo_description, seo_keywords	String
`payment_`	payment_methods, payment_bank_details	JSON / String
`notif_`	notif_whatsapp, notif_email_orders	'0' or '1'
`custom_`	custom_themes, custom_css	JSON / String

CHAPTER 3

Core PHP Architecture

config.php, functions.php helper reference, session system, layout files

3.1 config.php — Constants & DB Connection

Located at `includes/config.php`. Defines all application constants and the `getDB()` PDO singleton.

Constant	Value	Purpose
DB_HOST	localhost	MySQL host
DB_NAME	stockora	Database name
APP_NAME	Stockora AI	Application display name
APP_VERSION	2.0.0	Version string
CURRENCY	PKR	Default currency code
PKR_SYMBOL	Rs.	Currency symbol for display
SUBSCRIPTION_PRICE	10000	Monthly base plan in PKR
BASE_URL	Auto-detected	Full URL (http://host:port)
UPLOAD_PATH	assets/uploads/	Server path for uploads
SESSION_TIMEOUT	3600	Session cookie lifetime (seconds)

```
// PDO Singleton — call getDB() anywhere, returns same connection function getDB(): PDO {
static $pdo = null; if ($pdo === null) { $dsn =
sprintf('mysql:host=%s;port=%s;dbname=%s;charset=%s', DB_HOST, DB_PORT, DB_NAME, DB_CHARSET);
$pdo = new PDO($dsn, DB_USER, DB_PASS, [ PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC, PDO::ATTR_EMULATE_PREPARES => false, ]); }
return $pdo; }
```

3.2 functions.php — Helper Function Reference

Function	Signature	Description
`startSession()`	`void`	Starts PHP session with secure cookie params if not started
`isShopLoggedIn()`	`bool`	Returns true if shop session is valid
`isAdminLoggedIn()`	`bool`	Returns true if admin session is valid
`requireShop()`	`void`	Redirects to login if not authenticated; checks subscription
`requireAdmin()`	`void`	Redirects to login if not admin; superadmin role required
`getCurrentShop()`	`array`	Returns full shop row from DB for current session
`getDB()`	`PDO`	Returns PDO singleton (defined in config.php)
`sanitize(string)`	`string`	`htmlspecialchars(trim($input), ENT_QUOTES, 'UTF-8')`
`safeInt($val)`	`int`	Cast to int; returns 0 if not numeric
`safeFloat($val)`	`float`	Cast to float; returns 0.0 if not numeric
`formatCurrency(float)`	`string`	Returns "Rs. 12,345.00" formatted string
`jsonResponse(array, int)`	`void`	Sets JSON headers, ob_clean(), echo json, exit()
`getShopSetting(shopId, key, default)`	`string`	Reads one setting from settings table
`setShopSetting(shopId, key, value)`	`void`	Upserts one setting in settings table
`hashPassword(string)`	`string`	bcrypt hash via password_hash()
`verifyPassword(string, hash)`	`bool`	password_verify() — never use MD5/SHA1
`generateInvoiceNo(shopId)`	`string`	Generates sequential INV-XXXXX for shop
`uploadLogo(file, prefix)`		

Function	Signature	Description
	`string|false`	Validates and moves uploaded image; returns relative path
`redirect(url, msg, type)`	`void`	Header redirect with optional flash message
`flashMessage()`	`void`	Renders and clears session flash message HTML
`getLowStockProducts(shopId)`	`array`	Returns products where stock_qty <= reorder_level
`getShopDashboardStats(shopId)`	`array`	Returns today's sales, orders, product count etc.
`getAutopilotInsights(shopId)`	`array`	Runs AI analysis, returns insights array
`getProductEngineData(shopId)`	`array`	Returns winners/losers/hidden gems classification
`getBrandScore(shopId)`	`array`	Returns 0-100 score, grade, breakdown
`getSalesChartData(shopId)`	`array`	Returns 7-day daily sales data for Chart.js

3.3 Session & Authentication System

Two independent session namespaces exist side-by-side in the same PHP session:

Session Key	Portal	Set When
`$_SESSION['shop_id']`	Shop	On successful shop user login
`$_SESSION['user_id']`	Shop	On successful shop user login
`$_SESSION['user_role']`	Shop	owner / manager / cashier
`$_SESSION['shop_name']`	Shop	Cached shop name for display
`$_SESSION['admin_id']`	Admin	On successful admin login
`$_SESSION['admin_role']`	Admin	superadmin

□Session Fixation Prevention

After any successful login, call `session_regenerate_id(true)` to regenerate the session ID and prevent session fixation attacks. This is already done in `login.php`.

3.4 shop_layout.php & admin_layout.php

These files provide the full HTML shell (head, sidebar, topbar, footer) for each panel.

```
// Usage in any shop page: require_once '../includes/shop_layout.php'; shopHeader('Page Title', 'nav_active_key'); // ... page HTML ... shopFooter();
```

The **nav_active_key** highlights the correct sidebar menu item. Valid keys include: `dashboard`, `pos`, `products`, `categories`, `sales`, `purchases`, `expenses`, `customers`, `analytics`, `store_customize`, `theme_marketplace`, `settings`, etc.

CHAPTER 4

AJAX Contract & API Patterns

Standard JSON response format, writing AJAX handlers, frontend fetch pattern

4.1 Standard JSON Response Contract

Every AJAX response from any Stockora PHP file must follow this exact JSON structure:

```
// ▫ Success response { "success": true, "message": "Operation completed successfully", "data": { ... } // optional — any result payload } // ▫ Error response { "success": false, "message": "Descriptive error message" }
```

4.2 Writing a New AJAX Handler

All AJAX handlers live inside the same PHP file that renders the page, protected by a POST check at the top:

```
// ── AJAX Handler Block (place ABOVE shopHeader call) ────────── if ($_SERVER['REQUEST_METHOD'] === 'POST' && !empty($_POST['action'])) { ob_clean(); // CRITICAL: flush any partial HTML output $action = sanitize($_POST['action'] ?? ''); $db = getDB(); if ($action === 'save_item') { // 1. Validate required inputs $name = sanitize($_POST['name'] ?? ''); if (empty($name)) { jsonResponse(['success' => false, 'message' => 'Name is required'], 400); } // 2. DB operation (always prepared statement) $stmt = $db->prepare( "INSERT INTO items (shop_id, name) VALUES (?, ?)" ); $stmt->execute([$shopId, $name]); // 3. Return standardized response jsonResponse([ 'success' => true, 'message' => 'Item saved successfully', 'data' => ['id' => $db->lastInsertId()] ]); } // Unknown action — send 400 jsonResponse(['success' => false, 'message' => 'Unknown action'], 400); }
```

4.3 Frontend apiCall() Pattern

All frontend JS uses a standard async wrapper for consistency and DRY error handling:

```
/** * Standard AJAX helper — sends POST, returns parsed JSON * @param {string} action — maps to $_POST['action'] in PHP * @param {Object} params — additional form fields */ async function apiCall(action, params = {}) { const fd = new URLSearchParams({ action, ...params }); try { const res = await fetch(window.location.pathname, { method: 'POST', headers: { 'Content-Type': 'application/x-www-form-urlencoded' }, body: fd.toString() }); const json = await res.json(); if (!json.success) throw new Error(json.message || 'Unknown error'); return json; } catch (e) { console.error('apiCall error:', e.message); throw e; } } // Usage example: const result = await apiCall('save_item', { name: 'Widget A', price: '150' }); console.log(result.data.id); // New record ID
```

CHAPTER 5

Theme Engine

Architecture, how themes are stored, how to add new themes, preview URL

5.1 Theme Architecture Pipeline

The Stockora Theme Engine is a **pure data transformation pipeline**. No template files are duplicated per theme. Instead:

```
// 1. Theme name is stored in settings table: // shop_id | setting_key | setting_value // 42 | store_theme | "Luxury Fashion" // 2. store.php reads the active theme name: $activeTheme = getShopSetting($shopId, 'store_theme', 'Enterprise X'); // 3. Looks up the theme config in $themeMap (25 entries): $cfg = $themeMap[$activeTheme] ?? $themeMap['Enterprise X']; // 4. Injects CSS custom properties into <style> in <head>: <style>:root { --bg: <?= $cfg['bg'] ?>; --card-bg: <?= $cfg['card'] ?>; --nav-bg: <?= $cfg['nav'] ?>; --accent: <?= $cfg['acc'] ?>; --accent-2: <?= $cfg['acc2'] ?>; --text: <?= $cfg['txt'] ?>; --text-2: <?= $cfg['txt2'] ?>; --text-3: <?= $cfg['txt3'] ?>; --border: <?= $cfg['border'] ?>; }</style> // 5. ALL storefront CSS uses var(--token) — not hardcoded colors. // Result: changing theme = changing CSS variables = entire store re-themed.
```

5.2 Adding a New Theme

Adding a new theme requires editing **only one place** — the `$themeMap` array in `store.php`:

```
// In store.php — $themeMap array — add your new entry: 'My New Theme' => [ 'bg' => '#1a0a2e', // Page background 'card' => '#2d1256', // Card/section background 'nav' => '#0d0520', // Navigation bar background 'hero_from' => '#1a0a2e', // Hero gradient start 'hero_to' => '#0d0520', // Hero gradient end 'acc' => '#C4B5FD', // Primary accent color 'acc2' => '#F59E0B', // Secondary accent 'txt' => '#f5f3ff', // Primary text 'txt2' => 'rgba(245,243,255,.55)', // Secondary text 'txt3' => 'rgba(245,243,255,.25)', // Muted text 'border' => 'rgba(196,181,253,.1)', // Border color 'is_light' => false // true for light themes ],
```

> **That's All You Need**
>
> After adding the entry to `$themeMap`, the new theme automatically: (1) Appears in Theme Marketplace with a card (2) Works in Live Preview via `?preview_theme=My New Theme` (3) Can be installed, duplicated, backed up, exported — all theme management features work instantly.

Also add the theme name to the `$themes` array in `theme_marketplace.php` under the correct industry category.

5.3 Preview Theme URL Parameter

The `?preview_theme=ThemeName` URL parameter was added to `store.php` to enable the Live Preview iframe in the Theme Marketplace. Here's how it works:

```
// In store.php — after the isPreview check: if (!empty($_GET['preview_theme'])) {
$settings['store_theme'] = trim($_GET['preview_theme']); $isPreview = true; // Show full store
$notLaunched = false; // Bypass "store not launched" wall }
```

The marketplace uses this in its JavaScript live preview function:

```
function openPreview(themeName) { const iframe = document.getElementById('previewFrame');
const slug = currentStoreSlug; // from PHP echo iframe.src = `/store.php?s=${slug}
&preview_theme=${encodeURIComponent(themeName)}`; }
```

CHAPTER 6

Frontend Architecture

JavaScript IIFE modules, CSS design tokens, admin dark theme

6.1 JavaScript Module Pattern (IIFE)

Stockora uses the **IIFE (Immediately Invoked Function Expression)** module pattern for all frontend JS. No jQuery. No `var`. ES6+ only.

```
// Cart IIFE Module (in shop's page <script> block) const Cart = (() => { // ── Private state ──────────────────── let items = []; // ── Private functions ──────────────── function saveCart() { localStorage.setItem('stockora_cart', JSON.stringify(items)); } function loadCart() { const raw = localStorage.getItem('stockora_cart'); items = raw ? JSON.parse(raw) : []; } // ── Public API ─────────────────────── function addItem(product) { const existing = items.find(i => i.id === product.id); if (existing) existing.qty++; else items.push({ ...product, qty: 1 }); saveCart(); renderCart(); } function renderCart() { /* update cart DOM */ } // Auto-load on init document.addEventListener('DOMContentLoaded', loadCart); return { addItem, renderCart }; // expose public methods only })(); // Call it: Cart.addItem({ id: 5, name: 'Widget', price: 100 });
```

Forbidden Patterns in Stockora JS

- `var x = ...` — use `const` or `let` only
- `jQuery / $(...)` — use vanilla DOM APIs
- `innerHTML = unsanitized_user_input` — XSS risk
- Event listeners in HTML attributes (`onclick="..."`) for complex logic — use addEventListener
- `document.write()` — blocked

6.2 CSS Custom Properties System

The storefront uses **CSS custom properties (variables)** exclusively for all color values. This is what makes one-click theme switching possible.

```
/* ── Core tokens injected by PHP from $themeMap ── */ :root { --bg: #020812; /* page background */ --card-bg: #050e1c; /* cards, panels */ --nav-bg: #01050d; /* navigation bar */ --accent: #2563EB; /* buttons, links, highlights */ --accent-2: #60A5FA; /* secondary accent */ --text: #eff6ff; /* primary text */ --text-2: rgba(239,246,255,.55); /* secondary text */ --text-3: rgba(239,246,255,.25); /* muted / placeholder */ --border: rgba(37,99,235,.1); /* borders, dividers */ } /* ── Usage in component CSS (NEVER hardcode hex here) ── */ .product-card { background: var(--card-bg); border: 1px solid var(--border); color: var(--text); } .btn-primary { background: var(--accent); color: var(--bg); }
```

6.3 Admin Dark Panel Theme

The admin panel uses a **dark navy theme** applied under the `.app-wrapper` class scope — this ensures Bootstrap 5 components are not globally overridden.

```
/* In shop_layout.php / admin_layout.php <style> block: */ .app-wrapper { --bg: #0d1829; /* dark navy page background */ --card-bg: #162236; /* card background */ --sidebar-bg: #0a1220; /* sidebar */ --accent: #3b82f6; /* blue accent */ --text: #e2e8f0; /* light text on dark */ background: var(--bg); color: var(--text); min-height: 100vh; }
```

CHAPTER 7

Security Architecture

SQL injection, XSS, session security, file upload — rules and patterns

7.1 SQL Injection Prevention

Absolute Rule: ALL DB Queries MUST Use Prepared Statements

Never concatenate user input into SQL strings. Always use PDO prepared statements with bound parameters.

```
// ❌ NEVER DO THIS — SQL injection vulnerability: $db->query("SELECT * FROM products WHERE name = '" . $_POST['name'] . "'"); // ✅ ALWAYS DO THIS — parameterized query: $stmt = $db->prepare("SELECT * FROM products WHERE name = ? AND shop_id = ?");
$stmt->execute([sanitize($_POST['name']), $shopId]);
```

7.2 XSS Prevention

Context	Function to Use	Example
HTML output (PHP → HTML)	`htmlspecialchars()`	`<?= htmlspecialchars($name) ?>`
Form input value	`sanitize()`	`<input value="<?= sanitize($val) ?>">`
JS variable from PHP	JSON encode	`var x = <?= json_encode($val) ?>;`
DOM text (JS)	`element.textContent`	Never use `innerHTML` with user data
URL parameter in PHP	`urlencode()`	When building redirect URLs

7.3 Session Security

- **Session cookies** are set with `httponly: true` and `samesite: Lax`
- **Session regeneration** — `session_regenerate_id(true)` after every login to prevent session fixation
- **Session timeout** — 3600 seconds (1 hour) inactivity auto-logout
- **Role check** — every admin page calls `requireAdmin()`; every shop page calls `requireShop()`
- **shop_id source** — ALWAYS `$_SESSION['shop_id']`, never from URL/POST

7.4 File Upload Security

All file uploads go through `uploadLogo()` in functions.php which enforces:

```
// Validation inside uploadLogo(): $allowedTypes = ['image/jpeg', 'image/png', 'image/gif', 'image/webp']; $maxSize = 2 * 1024 * 1024; // 2MB limit // Check MIME type (not just extension) $finfo = new finfo(FILEINFO_MIME_TYPE); $mime = $finfo->file($file['tmp_name']); if (!in_array($mime, $allowedTypes)) { return false; // Reject non-image files } // Use random filename (never user-supplied filename) $newName = $prefix . '_' . bin2hex(random_bytes(8)) . '.jpg'; move_uploaded_file($file['tmp_name'], UPLOAD_PATH . $newName);
```

> ⚠ **Upload Directory Security**
>
> The `assets/uploads/` directory should have an `.htaccess` that prevents PHP execution: `AddHandler text/plain .php .php3 .phtml`

CHAPTER 8

Adding New Features — Workflow

Step-by-step guide to add new modules, DB columns, and navigation items

8.1 Adding a New Module (Page)

1. **Create the PHP file**

   Create `shop/my_feature.php` — start with the standard boot sequence (require functions.php → requireShop → AJAX block → data fetch → layout header → content → layout footer).

2. **Add Route to Sidebar**

   Edit `includes/shop_layout.php` — add a new `<a href="/shop/my_feature.php">` entry in the appropriate sidebar section.

3. **Define Navigation Key**

   Add the nav key to the active class logic in shop_layout.php so your page gets highlighted correctly.

4. **Create Any Required DB Tables**

   Write the CREATE TABLE SQL in database.sql (for documentation) and execute on dev DB.

5. **Add Helper Functions**

   If the feature has reusable data-fetching logic, add helper functions to functions.php with full PHPDoc.

```
<?php // shop/my_feature.php — Standard module template require_once '../includes/functions.php'; requireShop(); $shopId = (int)$_SESSION['shop_id']; $db = getDB(); // ── AJAX handlers ──────────────────────────────── if ($_SERVER['REQUEST_METHOD'] === 'POST' && !empty($_POST['action'])) { ob_clean(); $action = sanitize($_POST['action']); if ($action === 'save') { // ... handle save ... jsonResponse(['success' => true, 'message' => 'Saved!']); } jsonResponse(['success' => false, 'message' => 'Unknown action'], 400); } // ── Page data ──────────────────────────────────── $pageTitle = 'My Feature'; require_once '../includes/shop_layout.php'; shopHeader($pageTitle, 'my_feature'); ?> <div class="container-fluid py-4"> <!-- Your HTML here --> </div> <?php shopFooter(); ?>
```

8.2 Adding a Database Column / Table

1. **Write ALTER / CREATE SQL**

   Write the migration SQL. Always use `IF NOT EXISTS` / `IF NOT COLUMN EXISTS` to make it rerunnable.

2. **Update database.sql**

   Add the statement to `database.sql` so it's included in fresh installs.

3 **Update Type Definitions**

If using the settings table, document the new key in the settings key convention table (Chapter 2.4).

> ⚠ **Settings Table vs. New Column Rule**
>
> **Use the settings table** for per-shop configuration values (strings, JSON, toggles).
> **Add a new column** only when the data needs to be filtered/sorted/joined in SQL queries.
> Never add a column to the `shops` table for what should live in `settings`.

8.3 Adding a Sidebar Menu Item

Edit `includes/shop_layout.php`. Find the `<nav>` section and add your item:

```
<!-- In shop_layout.php sidebar nav --> <li class="nav-item"> <a class="nav-link <?= $activeNav === 'my_feature' ? 'active' : '' ?>" href="/shop/my_feature.php"> <i class="bi bi-star me-2"></i> <!-- Bootstrap Icon --> My Feature </a> </li>
```

CHAPTER 9

Commerce Cloud & Store Engine

store.php architecture, theme_marketplace internals, settings as config store

9.1 store.php Architecture

`store.php` is the **public storefront** — the only PHP file accessible without authentication. It resolves the shop from the `?s=slug` URL parameter.

```
// store.php — Top-level flow: // 1. Resolve shop from slug (settings table, NOT shops table) $slug = sanitize($_GET['s'] ?? ''); $shopId = findShopBySlug($slug); // queries settings WHERE setting_key='store_slug' // 2. Load all store settings at once (single query, cached in array) $settings = getAllStoreSettings($shopId); // 3. Check if store is launched $notLaunched = $settings['store_status'] !== 'active'; // 4. Handle preview_theme parameter (marketplace live preview) if (!empty($_GET['preview_theme'])) { $settings['store_theme'] = trim($_GET['preview_theme']); $isPreview = true; $notLaunched = false; } // 5. Resolve theme config from $themeMap $activeTheme = $settings['store_theme'] ?? 'Enterprise X'; $cfg = $themeMap[$activeTheme] ?? $themeMap['Enterprise X']; // 6. Handle AJAX cart/order actions (POST) if ($_SERVER['REQUEST_METHOD'] === 'POST') { ob_clean(); // handle: add_to_cart, place_order, etc. jsonResponse([...]); } // 7. Load products, categories, featured items // 8. Inject CSS custom properties + render full storefront HTML
```

9.2 theme_marketplace.php Architecture

`shop/theme_marketplace.php` is a single-file full-stack application containing:

Section	Location in File	Description
AJAX Handler Block	Top of file (before layout)	Handles 10+ actions via POST
Page Data	After AJAX block	Loads themes list, installed theme, backups, history
Layout + CSS	shopHeader() + <style>	Marketplace-specific CSS classes (.tm-*)
Theme Grid HTML	Page body	25 theme cards with live preview + install buttons
Preview Modal	Page body	Full-screen iframe modal for live preview
Version History Modal	Page body	Shows backups + install history

Section	Location in File	Description
JavaScript	Bottom of file	All interactive functions (installTheme, duplicate, etc.)

Complete AJAX Action Reference

action Value	What It Does	Key Settings Modified
install_theme	Saves new theme, backs up previous to rollback	store_theme, store_theme_rollback
rollback_theme	Restores store_theme_rollback value	store_theme
duplicate_theme	Copies theme entry to custom_themes JSON	custom_themes
delete_theme	Removes entry from custom_themes JSON	custom_themes
export_theme	Returns all store_* settings as JSON download	—
import_theme	Parses JSON, applies store_* settings, backs up current	all store_* keys
backup_theme	Saves snapshot of all store_* settings (max 10 kept)	theme_backups
restore_backup	Restores a specific backup by ID	all store_* keys
get_version_history	Returns theme_version_history + theme_backups arrays	—
record_history	Appends log entry to theme_version_history	theme_version_history

9.3 Settings Table as Config Store

The **settings table** is the backbone of the entire Commerce Cloud. Every theme feature uses it:

```
// Read a setting: $theme = getShopSetting($shopId, 'store_theme', 'Enterprise X'); // Write a setting (UPSERT — creates or updates): setShopSetting($shopId, 'store_theme', 'Luxury Fashion'); // Read complex JSON setting: $raw = getShopSetting($shopId, 'theme_backups', '[]'); $backups = json_decode($raw, true) ?: []; // Modify and save JSON setting: $backups[] = ['id' => uniqid(), 'theme' => $theme, 'ts' => time()]; if (count($backups) > 10) $backups = array_slice($backups, -10); setShopSetting($shopId, 'theme_backups', json_encode($backups));
```

CHAPTER 10

Deployment, Performance & Code Standards

Local setup, production checklist, performance rules, coding standards

10.1 Local Development Setup

1. **Install PHP 8.0+ & MySQL 8.0+**

   Install on your OS or use XAMPP / Laragon / Homebrew. Verify: `php --version` and `mysql --version`.

2. **Create Database**

   `mysql -u root -p -e "CREATE DATABASE stockora CHARACTER SET utf8mb4;"`

3. **Import Schema**

   `mysql -u root -p stockora < database.sql`

4. **Configure credentials**

   Edit `includes/config.php` — update DB_USER, DB_PASS, DB_NAME for your environment.

5. **Start Dev Server (PM2 Method)**

   Run: `pm2 start ecosystem.config.cjs` — starts PHP built-in server on port 3000.

6. **Access Application**

   Open browser: `http://localhost:3000`. Default admin: `admin@stockora.com` / `admin123`.

```
// ecosystem.config.cjs — PM2 config for dev server module.exports = { apps: [{ name: 'stockora', script: 'php', args: '-S 0.0.0.0:3000 -t /home/user/stockora', watch: false, instances: 1 }] }
```

10.2 Production Deployment Checklist

#	Task	Command / Action
1	Set display_errors = 0	`ini_set('display_errors', 0)` in config.php □already done
2	Enable HTTPS	Install SSL certificate; force HTTPS in .htaccess

#	Task	Command / Action
3	Change default admin password	Login → Admin → Settings → Change Password
4	Set secure session cookie	Add `'secure' => true` in session_set_cookie_params
5	Protect .htaccess	Deny access to includes/, database.sql, ecosystem.config.cjs
6	Set file permissions	`chmod 755` on directories, `chmod 644` on PHP files
7	Uploads directory only — no PHP execution	Create assets/uploads/.htaccess: deny PHP execution
8	Enable MySQL slow query log	For performance monitoring in production
9	Configure PHP error logging	Set error_log path in php.ini; monitor regularly
10	Set up daily DB backup	Cron: `mysqldump stockora | gzip > backup_$(date +\%F).sql.gz`

10.3 Performance Rules

Rule	Implementation
Single DB connection	PDO singleton in config.php — getDB() reuses same connection
Batch queries, not N+1	One JOIN query instead of a query per product in a loop
Index all WHERE / JOIN columns	shop_id, status, created_at, sku — all indexed in schema
Settings batch load	Load all settings in one query at page start; read from array, not per-key DB calls
Output buffering for AJAX	ob_clean() before JSON response to prevent HTML contamination
Paginate large datasets	Sales history, products list — always LIMIT + OFFSET; never SELECT * without limit
Image optimization	Resize uploads to max 800×800px server-side before saving
CDN for libraries	Bootstrap, Chart.js, SortableJS served from CDN — not bundled

10.4 Code Standards (PHP + JS + CSS)

PHP — PSR-12 Based

- 4 spaces indentation (no tabs)
- Opening brace on same line for functions/control structures
- Type declarations on all function signatures: `function foo(int $id): array`
- PHPDoc on all helper functions in functions.php
- All DB queries use PDO prepared statements — no exceptions
- Output escaping: always `htmlspecialchars()` when echoing user data to HTML
- Password hashing: always `password_hash()` / `password_verify()` — never MD5/SHA1

JavaScript — ES2020+

- `const` by default; `let` only when reassignment is needed; never `var`
- Arrow functions for callbacks; named functions for modules
- Async/await over .then() chains for readability
- IIFE module pattern — no global function pollution
- Event delegation for dynamic DOM: `document.addEventListener('click', e => { if (e.target.matches('.btn')) })`
- Never use `innerHTML` with user-supplied data (XSS risk)
- All fetch() calls must handle both network errors and JSON `success: false`

CSS — Custom Properties First

- Storefront: ALL colors must use `var(--token)` — never hardcoded hex values
- Admin panel: scoped under `.app-wrapper` to avoid Bootstrap conflicts
- BEM-like naming for storefront components: `.product-card__title`, `.nav-menu__item`
- Marketplace CSS uses `.tm-` prefix for all marketplace-specific classes
- No `!important` except for utility overrides in admin CSS
- Mobile-first media queries for storefront; desktop-first acceptable for admin

Git Commit Message Convention

```
# Format: <type>: <subject> feat: Add export to CSV for sales history fix: Resolve slug query fatal error in store_customize refactor: Extract formatCurrency into functions.php docs: Add Theme Engine section to developer guide perf: Replace N+1 product query with single JOIN security: Add session regeneration on login
```

Quick Reference — Most-Used Commands

`pm2 start ecosystem.config.cjs` — Start PHP dev server

`pm2 logs stockora --nostream` — View server logs

`pm2 restart stockora` — Restart after config changes

`mysql -u root -p stockora < database.sql` — Reimport schema

`php -l shop/my_file.php` — PHP syntax check

`git log --oneline -10` — View last 10 commits

`grep -r "function_name" --include="*.php" .` — Search codebase

Stack: PHP 8.0+ · MySQL 8.0+ · PDO · Bootstrap 5.3 · Vanilla JS ES2020+ · PM2